

Cloud Platform for Manufacturing Operational Data

An architecture proposal for a multi-tenant SaaS platform that collects data from PLCs and machine controllers at customer sites and serves it back as secure, per-customer dashboards and reporting.

Document	Manufacturing Operational Data (MOD) platform — architecture proposal
Version	2.3 — thin ingest API over Event Hubs; collector lifecycle, administration portal and optional command channel
Revision note	This variant replaces the managed device gateway of an earlier version with a thin ingest API in front of a durable log. The change follows from re-reading the requirements: they specify one-way communication, site to cloud, and one information type, operational data from PLCs and machine controllers. Everything that justified a device gateway was inferred from the operational requirements rather than stated (see §2.2 and §8.1). Version 2.3 shows the optional command channel and the administration portal in the reference architecture and describes the delivery pipeline in its own section (§6.4).
Fidelity	Diagrams are at whiteboard fidelity: they are intended to carry the reasoning, not to be production drawings.
Structure	Sections 1 to 8 cover the requirement areas in order. Section 8 carries every assumption the design rests on and what would change if any of them is wrong.

NOTE

This is an outside-in proposal, written from the requirements alone, so several decisions turn on numbers not yet known: event rates, retention requirements, the state of any existing codebase and how the product will be sold. Those decisions are named, and each one is capable of changing the design rather than merely refining it.

Contents

- Overview4**
 - The problem in one paragraph 4
 - The proposed design, summarised 4
 - Constraints the design is derived from 4
 - Communication direction and information types 5
 - Quality attributes, in priority order..... 5
- 1. High-Level Architecture6**
 - 1.1 How data flows..... 6
 - 1.2 Component responsibilities 7
 - 1.3 Why this shape..... 8
 - 1.4 Architecture principles 8
 - 1.5 The edge collector..... 9
- 2. Technology Selection 10**
 - 2.1 Edge..... 10
 - 2.2 Ingestion 11
 - 2.3 One submission method..... 12
 - 2.4 Ingestion options compared..... 13
 - 2.5 Processing 13
 - 2.6 Storage 14
 - 2.7 Serving and identity 15
 - 2.8 Platform and observability 16
- 3. Multi-Tenant Strategy 17**
 - 3.1 Tenancy model 17
 - 3.2 A hierarchy, not a flat tenant list..... 17
 - 3.3 Isolating customer data, layer by layer..... 18
 - 3.4 Securing tenant access 18
 - 3.5 Scaling as customers are added 18
 - 3.6 Managing tenancy in the application and data layers..... 19
- 4. Security & Compliance..... 20**
 - 4.1 Authentication and authorisation 20
 - 4.2 Encryption 20
 - 4.3 Secret management..... 21
 - 4.4 Secure communication between components..... 21
 - 4.5 Customer access control and compliance posture 21
- 5. Reliability & Failure Handling..... 22**
 - 5.1 Internet connectivity is lost..... 22
 - 5.2 An edge component fails..... 23
 - 5.3 A cloud service becomes unavailable 23
 - 5.4 Message volume grows significantly 24
 - 5.5 A customer site generates invalid or unexpected data 24
 - 5.6 Patterns applied throughout..... 25

6. Operations & Support.....	26
6.1 Monitoring strategy	26
6.2 Alerting approach.....	26
6.3 Troubleshooting workflow.....	26
6.4 Deployment strategy and delivery pipeline	27
6.5 Logging and diagnostics.....	28
6.6 Collector lifecycle.....	28
6.7 The MOD administration portal	32
6.8 Optional command channel	34
7. Cost Considerations	36
7.1 Major cost drivers.....	36
7.2 Where costs grow over time.....	36
7.3 Design decisions that balance scalability and cost	36
7.4 Sizing method	37
7.5 Estimated ingest cost at installed-base scale	37
8. Open Questions & Assumptions	39
8.1 Assumptions made.....	39
8.2 Information needed from business stakeholders.....	40
8.3 Technical unknowns that could significantly change the design	41
Appendix A: Decision record	42
Appendix B: Suggested first increment	43

Overview

The problem in one paragraph

Machines at customer sites produce operational data continuously. That data must reach a cloud platform across networks that are locked down and sometimes offline, be attributed to the right customer, site, line and asset, be stored economically for years, and be served back through a browser so that each customer sees only their own data. The platform must scale as customers are added and must be supportable by a small engineering team.

The proposed design, summarised

A lightweight **collector** runs on each customer site. It reads the controllers read-only, converts every source into one versioned event format, and writes to a durable local buffer before forwarding over an **outbound-only** TLS connection. The site never accepts an inbound connection, and a network outage delays data instead of losing it.

In Azure, a **thin ingest API** authenticates the collector, checks the batch envelope and appends it to **Event Hubs**, a durable log that decouples arrival from processing and allows reprocessing. Workers on **Container Apps** de-duplicate, validate, attribute each event to the tenant hierarchy, and project minute, hour and shift **rollups** into **Azure SQL**, which is what dashboards read. Raw events are kept briefly in an append-optimised store and age out to Parquet in **ADLS Gen2**.

Customers sign in through **Entra External ID**, federated to their own identity provider, and a .NET API behind **Front Door** returns only their tenant's data — enforced in the application and again by row-level security in the database. Tenancy is pooled by default and dedicated by exception, with the isolation mode held as a property of the tenant rather than as a separate architecture. MOD staff administer tenants and collectors through a separate, audited administration portal; an optional command channel lets support ask a collector for diagnostics without opening anything on the plant network.

Operationally, the headline signals are per-site **freshness** and per-tenant **completeness**, not API uptime. Everything is deployed from Git through infrastructure as code, and the collector fleet is upgraded in reversible rings.

Constraints the design is derived from

#	Constraint	Consequence inherited by every decision below
C1	Plant networks do not accept inbound connections from the internet.	All communication is initiated from inside the customer network. Anything needing a per-site firewall exception is a deployment blocker.
C2	Machines keep producing whether or not the link is up (intermittent connectivity).	Capture cannot depend on connectivity. A durable local buffer is mandatory, not an optimisation.
C3	Data is read from PLCs and machine controllers without changing their programs.	Read-only access, gentle polling or subscriptions, no writes to production equipment, ever.
C4	A small engineering team builds and supports the platform.	Operational surface is a first-order design input: managed services unless there is a specific reason not to.
C5	Microsoft ecosystem (Azure, .NET, SQL Server).	First-party Azure services by default; portability retained only where it is cheap.

Communication direction and information types

The requirements are explicit on both, and the design follows them literally rather than assuming more.

Aspect	What the requirements specify	What this design does
Direction	Data is transmitted to the cloud; the only other direction named is cloud to browser. No cloud-to-site requirement appears anywhere.	The telemetry path is strictly one-way. No inbound connection to the site, and no persistent downlink channel by default.
Information types	Operational data from PLCs and machine controllers.	Machine states, counters, rates, fault codes and process values, carried as one canonical event. No writes to control systems, ever.
Implied management traffic	Not stated. It follows from the requirements that the platform be supportable by a small team, have monitoring and diagnostics built in, and be maintainable with modern practices.	Collector configuration, versions and health are handled by pull: the collector asks a management endpoint on its own schedule. This keeps the network model one-way and is recorded as an assumption (A13). An optional command channel, disabled unless a customer accepts it, is described in §6.8.
Beyond the requirements	Not requested: historian and file-drop sources, exports and integration APIs.	Included as fallbacks and near-term extensions, and flagged as such where they appear.

Quality attributes, in priority order

Architecture is the resolution of competing qualities. This is the ranking the design assumes; if the business ranks them differently, several decisions below should change.

- 1. Data integrity.** A wrong number is worse than a late number and far worse than a missing one. Availability and OEE figures are only as good as the completeness of the data behind them.
- 2. Tenant isolation.** Layered, so that no single failed control produces a cross-tenant leak.
- 3. Operability by a small team.** Every component must be diagnosable by people who also write features.
- 4. Freshness.** Latency is what customers feel, and staleness must be visible when it happens.
- 5. Cost per tenant.** With multi-year retention, unit economics become an architectural property.
- 6. Evolvability.** New equipment types, metrics and tenancy modes should be additive changes.
- 7. Query performance.** Achieved by projection into read models, not by tuning the write path.

1. High-Level Architecture

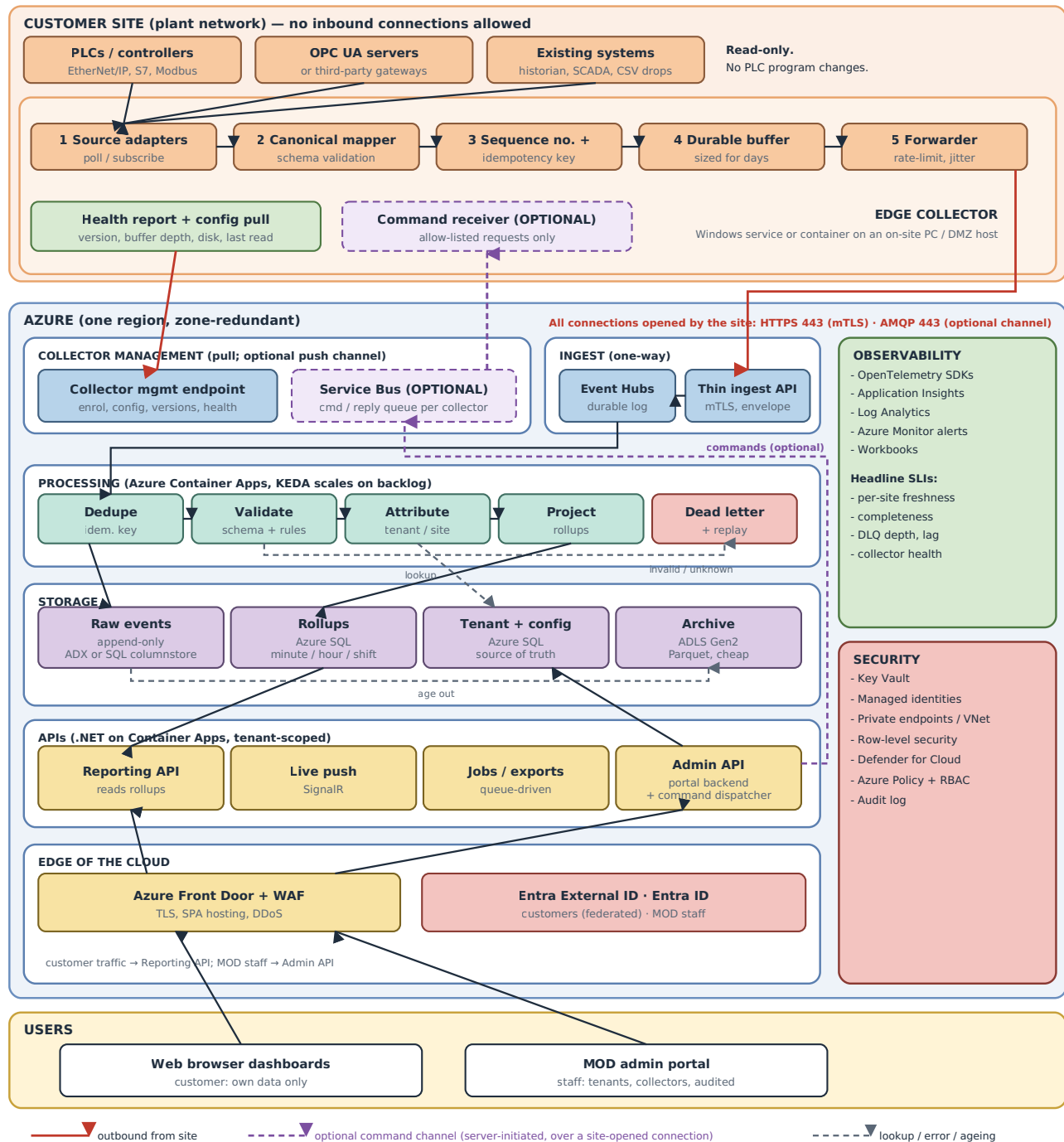


Figure 1. Reference architecture. Solid arrows are data flow; dashed arrows are lookups, error paths and ageing; purple dashed arrows are the optional command channel. Every connection to the cloud is opened by the site; nothing is pushed into the site. The MOD administration portal reaches the Admin API through Front Door. The administration portal and the command channel are detailed in §6.7 and §6.8; delivery is shown separately in §6.4.

1.1 How data flows

- 1. Read.** Source adapters read tags from PLCs, OPC UA servers or systems the site already runs, by polling or by subscription, always read-only.

- 2. Normalise.** Every source is mapped to one versioned canonical event and validated on site, so a misconfigured source is caught where it originates and everything downstream sees exactly one shape.
- 3. Label.** Each event receives an idempotency key and a per-source monotonic sequence number. The key lets the cloud discard duplicates; the sequence lets it detect gaps.
- 4. Buffer.** Events are appended to a durable local store that survives restarts and is sized for a multi-day outage.
- 5. Forward.** Batches are POSTed over outbound HTTPS to the ingest API, authenticated by the collector's own client certificate, with jittered retry and rate-limited catch-up. The buffer entry is deleted only after the API acknowledges.
- 6. Ingest.** A thin API authenticates the collector, stamps the batch with the authenticated collector identity, validates the envelope, appends it to Event Hubs and only then returns success. It does no business logic, so it stays stateless and cheap to scale. There is one submission method for everything: a batch, capped in event count and in bytes. In normal operation batches go out part-filled on a short timer; after an outage they go out full until the backlog clears (§2.3).
- 7. Process.** Container Apps workers de-duplicate, validate again, attribute each event to tenant, site, line and asset from the registry, and project rollups. Invalid events are dead-lettered with a replay path.
- 8. Store.** Raw events to an append-optimised store; rollups to Azure SQL; ageing raw data to Parquet in ADLS Gen2.
- 9. Serve.** A tenant-scoped .NET API reads rollups; SignalR pushes live updates; exports and scheduled reports run as queued jobs isolated from live ingestion.
- 10. Present.** Browser dashboards reach the platform through Front Door; users sign in via Entra External ID federated to their own identity provider. MOD staff use the administration portal, which reaches the Admin API through the same Front Door and signs in with corporate Entra ID.
- 11. Manage.** Collectors pull desired configuration and version from the management endpoint and post health reports. Where a customer enables it, the Admin API's command dispatcher can also send allow-listed requests over the optional command channel (§6.8).
- 12. Observe.** Every component emits OpenTelemetry signals to Azure Monitor; collectors report health in their health reports; alerts are driven by data freshness and completeness.

1.2 Component responsibilities

Component	Responsibility	Note
Edge collector	Read sources, map to canonical events, validate at the edge	Pluggable adapters; the equipment knowledge plugs in here
Local buffer	Durable append-only store surviving restart and multi-day outage	Sizing is an open question (§8)
Forwarder	Outbound TLS delivery, batching, rate-limited catch-up, jittered reconnect	Anti-herd behaviour is not optional at hundreds of sites
Ingest API	Authenticate the collector, stamp identity, validate the envelope, append to the log	Deliberately thin: no business logic, so it scales trivially and fails rarely
Event Hubs	Durable log between arrival and processing: replay, burst absorption, ordering per partition	Partitioned by collector; seven-day retention on Standard
Collector management endpoint	Serves enrolment, desired configuration, desired version and accepts health reports; brokers credentials for the optional command channel (§6.8)	Pull-only, so the site keeps a one-way network posture

Component	Responsibility	Note
Admin API	Backend of the MOD administration portal: tenants, hierarchy, collectors, desired state, audit; hosts the command dispatcher	Staff access only, through Front Door; every action audited
Command channel (optional)	Service Bus queue pair per collector; server-initiated requests over a connection the collector opened	Disabled unless the customer accepts it; the platform is fully operable without it
Processing workers	De-duplicate, validate, attribute, project rollups	Container Apps with KEDA on consumer lag
Dead letter + replay	Quarantine invalid events with an operator-visible replay path	Monitored, with an owner; a queue nobody watches is data loss
Raw event store	Append-optimised full-fidelity storage, partitioned by time	Engine choice deliberately deferred (§2.6)
Rollup store	Read-shaped aggregates at minute, hour and shift grain	Azure SQL; this is what reporting queries hit
Tenant and config registry	Hierarchy, entitlements, isolation mode, placement, device mapping	Azure SQL; source of truth for everything else
Cold archive	Parquet on ADLS Gen2 for recomputation and compliance	Out of the interactive query path
Reporting API / live push / jobs	Tenant-scoped reads, real-time updates, exports	.NET on Container Apps; SignalR; queued background work
Observability	Traces, logs, metrics, collector health, alerting	SLIs are per-site freshness and per-tenant completeness

1.3 Why this shape

Two structural decisions carry the design; nearly everything else follows from them plus the constraint that a small team has to operate it.

- **A durable buffer at the edge** turns connectivity loss from a data-loss problem into a freshness problem, which converts the intermittent-connectivity requirement from a risk into a sizing parameter.
- **Projecting raw events into rollups on ingest** means the read path never touches the write path. A heavy customer query cannot slow ingestion, the append-optimised store stays append-optimised, and years of retention stay affordable because rollups are far smaller than raw data.

1.4 Architecture principles

Principle	Meaning
The edge never waits for the cloud; the cloud never assumes the edge is reachable	Both sides operate correctly while disconnected and reconcile when the link returns.
One data model, many transports	An agent, a read from a system the customer already runs, and a scheduled file drop all land on the same canonical event, so a difficult site is a configuration decision rather than a custom project.
Compute centrally	Derived metrics are computed in the cloud so definitions can change and history can be recomputed.

Principle	Meaning
Isolation mode is a property, not an architecture	A tenant moves between pooled and dedicated without a redesign.
Project into query shapes	Do not tune the write path into a read path; build read models.
Assume every control eventually fails	For every guarantee, name the second control behind it.

1.5 The edge collector

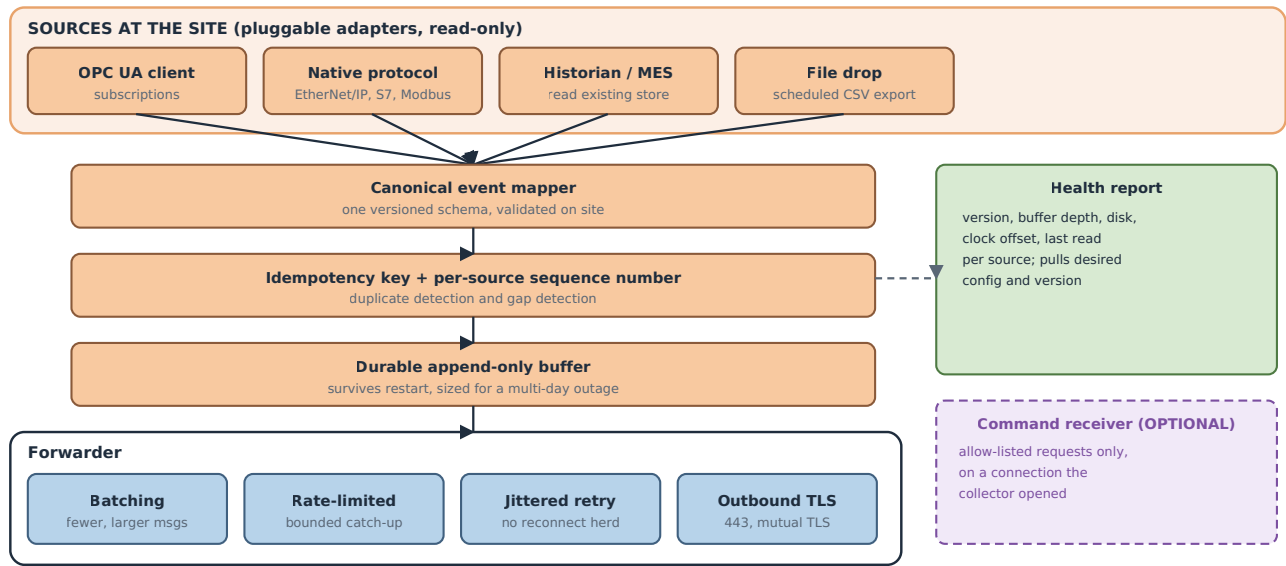


Figure 2. Edge collector internals. Adapters are pluggable; everything downstream of the mapper sees exactly one event shape.

- **Canonical event, minimum content:** stable event identity; source identity resolvable to the tenant hierarchy; event time and observed time as separate fields; event type; typed payload; quality flag; schema version. Consumers tolerate additive change.
- **Keys are assigned at the edge.** A key assigned on arrival in the cloud does not survive the retry that produced the duplicate. Sequence numbers also give gap detection, and a gap is the more dangerous of the two: incomplete data reported as complete is a wrong number rather than a missing one.
- **The forwarder, not the collector, owns flow control.** Rate-limited catch-up bounds the load a returning site can impose; jittered reconnect prevents a regional restoration from producing a synchronised stampede. The realistic failure mode at hundreds of sites is not one site returning, it is all of them returning at once.
- **Management is by desired state, pulled not pushed.** On a schedule the collector asks the management endpoint for its desired configuration and version, and posts a health report carrying version, buffer depth, disk, clock offset and last successful read per source. Nothing reaches into the site.
- **The command receiver is optional.** Where a customer enables the command channel, the collector also holds an outbound connection on which it accepts a small, allow-listed set of requests (§6.8). It never accepts an unsolicited instruction on any other path.

2. Technology Selection

The selection rule throughout is: managed services over self-operated ones, first-party Azure unless there is a specific reason otherwise, and defer any decision that depends on numbers we do not yet have while keeping it cheap to reverse. Each card below states the choice, the reasoning, the alternatives considered, the trade-off accepted and the evidence that would reopen it.

2.1 Edge

Collector runtime	
Choice	A .NET collector running as a Windows service, with a container image produced from the same codebase.
Why	Matches the team's language and the Windows hosts plant IT teams already manage; gives full control over buffering, idempotency and flow control, which are the parts that decide data integrity; and keeps the install light, which is the product's differentiator.
Alternatives considered	Azure IoT Edge — good cloud-managed module deployment, but requires a container runtime on site. Azure IoT Operations — powerful, with OPC UA connectors and an MQTT broker, but requires Kubernetes at the edge via Azure Arc: heavy for plant IT and for a small team. Third-party gateway products — quick to start, but a per-site licence and less control over integrity guarantees.
Trade-offs accepted	We own the collector and its update mechanism, including the fleet risk that comes with it.
Revisit if	Customers standardise on edge Kubernetes, or a gateway product already covers most sites.

Equipment access	
Choice	OPC UA client adapter first, using subscriptions where available; native protocol adapters, historian readers and a file-drop adapter behind the same interface.
Why	OPC UA is vendor-neutral, secure by design and supports report-by-exception, which lowers both PLC load and data volume. Pluggability matters because the set of source systems grows with the customer base rather than with the engineering roadmap.
Alternatives considered	Native protocols only (fast for one vendor, a treadmill across many); requiring every customer to install a gateway (adds cost and a dependency); MQTT/Sparkplug B only (clean, but needs something on site already publishing it).
Trade-offs accepted	More adapter code over time; contained by a strict adapter contract and a test harness driven by recorded traffic.

Edge-to-cloud transport	
Choice	HTTPS POST of compressed batches on port 443 with mutual TLS; configuration and versions pulled over the same port. The optional command channel uses AMQP over WebSockets on 443, on a connection the collector opens.
Why	Outbound 443 is the port plant firewalls are most likely to permit; plain HTTPS works through corporate proxies and needs no device SDK. The requirements name no downlink, so no persistent connection is held by default.

Edge-to-cloud transport

Alternatives considered	MQTT or AMQP through a device gateway (a persistent connection that also carries commands downward, but a two-way platform is not required); native MQTT on 8883 (efficient, frequently blocked); site-to-site VPN or ExpressRoute (strong, but a network project per site — offer it only where a customer demands it).
Trade-offs accepted	Per-request overhead compared with a persistent connection, contained by batching and compression.

2.2 Ingestion

The requirements specify uplink only, so the ingestion layer is two components rather than a managed device gateway: a thin API that authenticates collectors and validates envelopes, and a durable log that decouples arrival from processing. A third, separate endpoint serves collector enrolment and configuration on a pull basis.

Ingest endpoint: a thin HTTPS API on Container Apps

Choice	Collectors POST compressed batches over HTTPS, authenticated with a per-collector client certificate (mutual TLS) or a short-lived token obtained with it. The API validates the envelope, stamps the authenticated collector identity onto every event, appends the batch to the log and acknowledges.
Why	Keeps authentication, tenant attribution and rejection semantics in our own code, which matters because attribution from the authenticated identity is the control that prevents one customer's collector writing into another's data. Plain HTTPS on 443 is the most firewall-friendly option available, works through corporate proxies, and needs no device SDK. Synchronous rejection lets a misconfigured site be told at the door rather than through a dead-letter queue.
Alternatives considered	Azure IoT Hub — gives per-device identity, provisioning, twins, direct methods and update orchestration out of the box, but it is a two-way device platform and the requirements need one way. Its 4 KB message metering also makes it roughly five to ten times the cost of the log alone at full installed-base volume. Event Grid MQTT broker — per-client certificates and routing into Event Hubs, cheaper per message, no twins or update service. Writing to Event Hubs directly from the collector — fewest moving parts, but shared namespace credentials and no per-collector identity, which is not acceptable for tenant attribution.
Trade-offs accepted	We build and operate enrolment, certificate lifecycle, configuration distribution and update rollout ourselves: a few hundred lines and a handful of operational routines that a device platform would have provided.
Revisit if	Site count grows into the high hundreds and fleet lifecycle becomes the dominant operational cost, or a genuine downlink requirement appears: adopt IoT Hub for identity and control while keeping bulk telemetry on this path.

Durable log: Azure Event Hubs (Standard)

Choice	Every accepted batch is appended to Event Hubs, partitioned by collector, and processed from there by checkpointed consumers.
Why	Three things justify it, in order. Replay: the edge buffer is deleted on acknowledgement, so the log is the only second copy; a projection bug found a week later is recoverable with it and unrecoverable without it. Burst absorption: the characteristic failure here is hundreds of sites draining multi-day buffers at once, and the log lets the database be sized for average load instead of for that spike. Decoupling: a database failover becomes backlog rather than errors visible at every site. Ordering per partition and consumer checkpoints come with it.

Durable log: Azure Event Hubs (Standard)

Alternatives considered	Event Hubs Basic — half the price, but one consumer group and 24-hour retention, and it cannot be upgraded in place. A raw landing area in ADLS written by the API before acknowledgement — gives replay at storage prices, but ordering, checkpointing and consumer coordination become our code, and per-write transaction charges punish small, frequent batches. Storage or Service Bus queues — destructive reads, so no replay. Writing straight to the database — no replay, and the database must be permanently over-provisioned for catch-up bursts.
Trade-offs accepted	One more managed service in the path, and consumer parallelism is capped by partition count, so partitions are sized for the fleet we expect rather than the one we have.
Revisit if	Retention beyond seven days is needed (move to Premium), or a second independent consumer never materialises and one-day replay proves sufficient (drop to Basic).

Collector identity, enrolment and configuration

Choice	A certificate authority issues one client certificate per collector at install; a separate management endpoint serves desired configuration and desired version, accepts health reports, and handles certificate renewal. Collectors poll it; nothing is pushed into the site.
Why	Preserves the one-way network posture the requirements imply, gives per-collector revocation, and keeps the enrolment and configuration model in our own data store alongside the tenant registry, where the collector-to-asset mapping already lives.
Alternatives considered	Device twins in IoT Hub (the managed equivalent, with a push channel we do not need); manual configuration at install (unmanageable past a few dozen sites); a third-party remote management tool.
Trade-offs accepted	Configuration changes take effect within one polling interval rather than immediately, and we own the certificate lifecycle including expiry alerting.

NOTE

Partitioning: partition by source, never by tenant. Customer sizes are heavily skewed, so tenant partitioning creates hot partitions that cannot be scaled out of. Partitioning by collector gives per-site ordering; a site large enough to become hot is split into per-line collector identities. Cross-asset ordering is not required, and per-source sequence numbers carry ordering regardless.

2.3 One submission method

Everything a collector sends arrives the same way: a batch of canonical events, capped both in event count and in bytes so that no single request can degrade the ingest tier. The cap is sized for the largest plausible event rather than the average, and the same endpoint, the same authentication and the same acknowledgement semantics apply whether the batch carries four events or four hundred.

- **Normal operation sends part-filled batches on a short timer.** Freshness, not efficiency, sets the cadence: a batch goes when the timer expires or the cap is reached, whichever comes first.
- **After an outage the same loop sends full batches back to back until the backlog clears,** at a rate the forwarder caps so that a returning site cannot crowd out live traffic from other sites. Catch-up is therefore the ordinary path running at its ceiling, not a special mode.
- **Everything lands in the log by construction.** There is one arrival surface, one set of metrics, one idempotency store and one replay story, and any future subscriber sees backfilled periods and live traffic alike without anything having to be merged.

- **The cost is paid in the quiet periods.** Small batches carry proportionally more per-request overhead and compress less well than large ones, so the batch timer should be tuned against the freshness target rather than left at a default (§7.5).

NOTE

Two things deliberately not built, recorded here so the reasoning is on the record. A bulk upload path: a very large backlog could be uploaded to blob storage as one compressed file, with a worker republishing it into the log. It compresses better, resumes after a dropped link and turns thousands of requests into one. It is not proposed, because it adds a second authentication flow, a second set of metrics and a dual-source dedupe problem to solve a burst that a stateless ingest tier should absorb. It becomes worth revisiting if a single site's backlog is measured in gigabytes rather than megabytes. **Concurrent batches in flight per collector:** with one batch outstanding at a time, drain rate is bounded by round-trip latency rather than bandwidth, so a very large backlog on a high-latency link drains slowly. Allowing a small number of concurrent batches would fix that cheaply, at the cost of out-of-order arrival the dedupe and sequence logic already tolerates. Neither is specified here; both are named so that the limitation is a known one rather than an oversight.

2.4 Ingestion options compared

Indicative monthly figures for the full installed base under the volume assumptions in §7.5. They are list-price estimates and should be confirmed against the Azure pricing calculator before use.

Option	Indicative monthly	Absorbs bursts	What it gives up
Ingest API + Event Hubs Standard (proposed)	~\$45 for the log, plus ordinary container cost	Yes	Nothing material; one extra managed service
Ingest API + Event Hubs Basic	~\$22	Yes	24-hour retention, one consumer group, no in-place upgrade
Ingest API + blob landing area	~\$35 at coarse batching; far more at five-second batches	Yes	Ordering, checkpointing and consumer coordination become our code
Ingest API + queue	~\$50 to \$190	Yes	Destructive reads, so no replay
IoT Hub	~\$250 to \$500 at 4 KB metering	Yes	Nothing technically; pays for a downlink platform the requirements do not include
Ingest API writing straight to SQL	No streaming cost, but several hundred a month of permanent database headroom	Only by over-provisioning	Replay and decoupling

2.5 Processing

Compute: Azure Container Apps with KEDA

Choice	Container Apps hosts stream workers, APIs and jobs. Workers scale on consumer lag, not CPU.
Why	Containers without operating a Kubernetes cluster; revisions and traffic splitting give canary releases; scale to zero outside production. Scaling on CPU is wrong for consumers because a worker blocked on a slow dependency looks idle while backlog grows.

Compute: Azure Container Apps with KEDA	
Alternatives considered	AKS — full control, but the team would be running a cluster. Azure Functions — simple, but less control over batching, checkpointing and long-running work. Stream Analytics — quick for windowed aggregates, harder to test, version and express restatement in.
Trade-offs accepted	Less control than AKS; consumer parallelism is capped by partition count, so partitions are sized for growth.
Revisit if	A specific platform need appears; the workloads are ordinary containers, so moving to AKS is realistic.

2.6 Storage

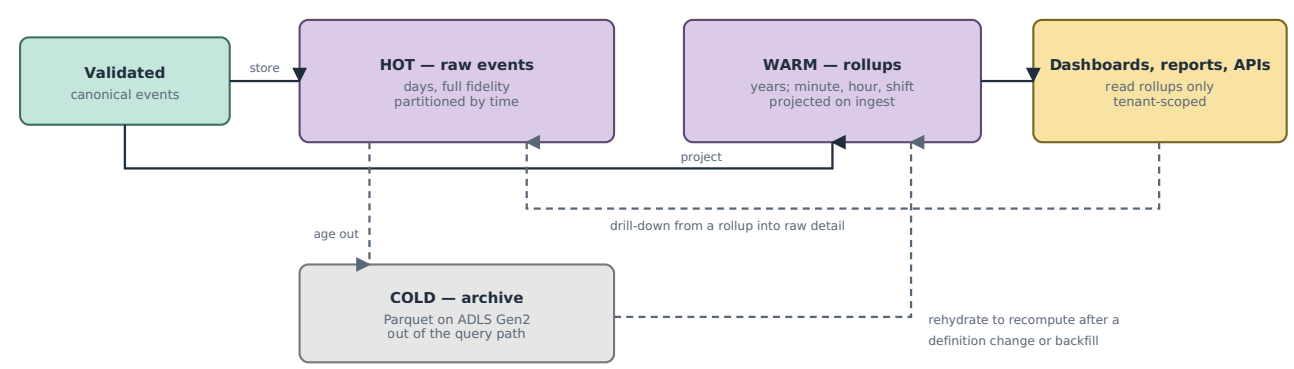


Figure 3. Storage tiers and query paths. Rollups are orders of magnitude smaller than raw events: this is what makes multi-year retention affordable and keeps reporting off the write path.

Rollups (read model): Azure SQL Database	
Choice	Minute, hour and shift aggregates per asset, shaped for dashboard queries, with a tenant key on every row and row-level security beneath the application filter.
Why	The team's core skill; mature row-level security; elastic pools make per-tenant databases affordable when needed; excellent .NET tooling. Rollup volume is modest, so a relational engine is sufficient.
Alternatives considered	Cosmos DB (global scale, higher cost, new skill for relational reporting); PostgreSQL (capable, but moves off the SQL Server stack); serving everything from the analytics engine (less natural for per-user, per-tenant reporting).
Trade-offs accepted	Vertical limits of a single database, addressed by elastic pools, Hyperscale and deployment stamps (\$3.5).

Raw events: decision deliberately deferred	
Choice	Choose between Azure Data Explorer and Azure SQL with clustered columnstore and partition switching after measuring real event rates at pilot sites; keep the projection layer behind an interface so the raw store stays swappable.
Why	ADX is the better engine for this query shape, with native ingestion, compression and retention policies. A partitioned columnstore table is materially easier for a SQL Server-fluent team to operate. This is a team-capability decision as much as a technical one and should be made against data.

Raw events: decision deliberately deferred

Alternatives considered	Committing now to either engine; time-series extensions on PostgreSQL; Fabric Eventhouse.
Trade-offs accepted	A little abstraction work in the projection layer, in exchange for not stranding the team on the wrong engine.
Revisit if	Measured volume exceeds what columnstore handles comfortably, or ad-hoc exploration over raw data becomes a product feature: choose ADX.

Archive: ADLS Gen2 with Parquet

Choice	Raw events older than the hot window are written as Parquet partitioned by tenant and date, with lifecycle rules moving them to cooler access tiers.
Why	Cheapest durable storage in an open format that can be read back for recomputation, backfills or compliance.
Alternatives considered	Keeping everything in the raw store (simple, expensive); blob storage without a hierarchical namespace.
Trade-offs accepted	Archive queries are slow and rehydration takes time; acceptable because it is not a dashboard path.

2.7 Serving and identity

Reporting API, Admin API and real-time updates

Choice	.NET minimal APIs on Container Apps, tenant-scoped by middleware, reading rollups; a separate Admin API for the MOD administration portal; Azure SignalR Service for live push to browsers; exports and scheduled reports as queued background jobs.
Why	One hosting model for everything; managed WebSocket fan-out without running a backplane; jobs isolated from live ingestion capacity so a large export cannot affect dashboards; staff operations separated from customer traffic.
Alternatives considered	App Service (equally valid for the API alone); API Management in front (worth adding when external API consumers and rate plans exist); client polling instead of SignalR (simplest, wasteful at scale).
Trade-offs accepted	No API gateway product on day one; rate limiting is done in the application and at Front Door.

Identity: Microsoft Entra External ID and Entra ID, plus Front Door with WAF

Choice	Customer users sign in through External ID, with enterprise customers federating their own identity provider; MOD staff sign in with corporate Entra ID. All browser and API traffic enters through Front Door Premium, which terminates TLS, applies WAF rules and reaches origins privately.
Why	Federation means the customer's own joiner-leaver process and MFA govern access. A single hardened public entry point keeps application origins off the public internet.
Alternatives considered	Azure AD B2C (the predecessor for customer identity); Auth0 or Okta (excellent, extra vendor and cost); Application Gateway instead of Front Door (regional, fine while single-region).

Identity: Microsoft Entra External ID and Entra ID, plus Front Door with WAF**Trade-offs accepted**

Some per-customer federation work at onboarding; Front Door Premium carries a fixed cost.

2.8 Platform and observability

Concern	Choice	Alternatives	Trade-off accepted
Secrets and service identity	Managed identities everywhere; Key Vault for what remains; OIDC workload identity federation from CI	Secrets in app settings or pipeline variables; HashiCorp Vault	Most secrets are eliminated rather than protected; a little extra wiring per service
Observability	OpenTelemetry into Application Insights, Log Analytics and Azure Monitor alerts	Datadog, Grafana Cloud, self-hosted Prometheus stack	Log ingestion is billed per GB, so a logging budget and sampling policy are required (\$7)

Infrastructure as code, CI/CD and collector fleet updates are described together in §6.4.

3. Multi-Tenant Strategy

3.1 Tenancy model

Model	Description	Why not / why yes
Silo	Every customer gets their own full stack.	Rejected as the default: operational surface multiplies, every schema change fans out, tenants drift, and idle capacity is paid for per customer. A small team cannot carry it at hundreds of sites.
Pool	All customers share everything, separated by a tenant key.	Rejected as the only model: cannot satisfy a customer whose contract or jurisdiction forbids shared infrastructure.
Bridge (proposed)	Pooled by default, dedicated by exception, with isolation mode held as a control-plane property.	Gives the economics of pooling with a credible path to dedicated infrastructure, and a tenant can move between the two without an application rewrite. Elastic pools make the dedicated end affordable.

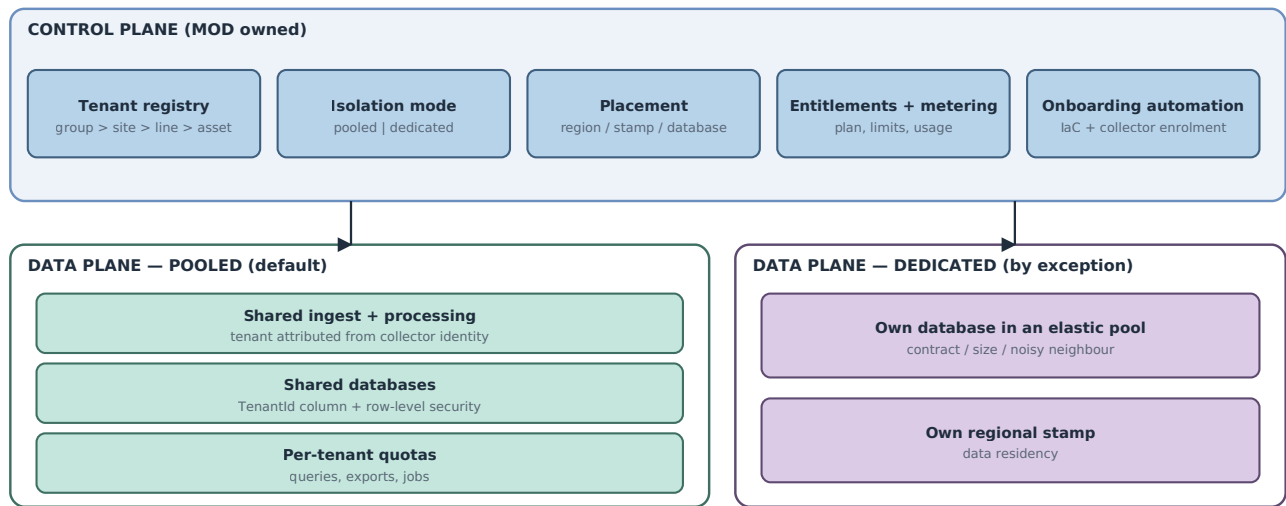


Figure 4. Control plane and data planes.

The control plane holds tenant identity, the customer hierarchy, entitlements, metering configuration, isolation mode, regional placement and the mapping from collectors to assets. The data plane serves workloads. Separating them is what makes onboarding automatable and what allows a tenant to change isolation mode without a rewrite.

3.2 A hierarchy, not a flat tenant list

A single customer is typically a corporate group with many sites, each with lines and assets, and different people entitled to different slices. Two consequences follow.

- **Attribution is captured at ingestion, at the finest level**, because hierarchy cannot be reconstructed reliably afterwards.
- **Rollups are materialised at the leaf and aggregated upward at query time**, because the tree changes: a line moves, a site is sold, a group reorganises. Storing only leaf rollups means reorganisation does not rewrite history.

3.3 Isolating customer data, layer by layer

Layer	Control
Identity	The tenant is resolved from the validated token and checked against the registry. It is never taken from a URL, header or request body.
Application	Tenant context middleware and global query filters in the data-access layer, so a developer cannot forget the tenant predicate.
Database (pooled)	Row-level security bound to the session's tenant value, filtering reads and blocking cross-tenant writes. This is defence in depth beneath the application filter, not a substitute for it.
Database (dedicated)	A separate database per tenant in an elastic pool, selected by the placement record. Same schema, same code.
Ingestion	Events are attributed from the authenticated collector identity, never from the payload, so a compromised or misconfigured collector cannot write into another customer's data. Unmapped sources are quarantined, never guessed.
Files, exports and archive	Tenant-scoped paths and short-lived, narrowly scoped access. Egress is where isolation most often fails in practice, because the export path tends to be built late and under pressure.
Real-time push	Group membership is authorised server-side after an entitlement check, never trusted from the client.
Command channel (optional)	One queue pair per collector, named with the tenant; each collector's credential is scoped to its own pair only.
Caches and logs	Tenant-prefixed cache keys; tenant identifiers on every log record.
Verification	Automated cross-tenant tests in CI: every endpoint called as tenant A using tenant B's identifiers must return not-found. A failure blocks the release.

3.4 Securing tenant access

- **Federated sign-in** so each customer's identity provider, and their MFA, governs who gets in and who loses access.
- **Roles scoped to hierarchy nodes** (for example Viewer, Supervisor, Tenant Admin at group, site or line level), inherited downward and checked for every requested asset.
- **Customer self-service administration**, so tenant admins manage their own users.
- **MOD staff access is separate and controlled:** corporate identity with MFA and conditional access, just-in-time elevation, a recorded reason, and an audit trail that can be exposed to the customer.

3.5 Scaling as customers are added

- **Scale out the shared plane first:** stateless APIs and workers scale horizontally; stream partitions and log throughput units are sized ahead of growth.
- **Scale the data tier:** add compute, move to Hyperscale, or move a demanding tenant to its own database in an elastic pool.
- **Deployment stamps:** when a set of shared resources approaches a comfortable limit, deploy an identical copy from the same infrastructure code and place new tenants there. Stamps also deliver regional residency and limit blast radius.
- **Per-tenant quotas** on query range, export size and concurrent jobs contain noisy neighbours.

- **Automated onboarding:** create the tenant, choose placement, provision dedicated resources if required, configure federation, issue enrolment tokens for the site's collectors, seed the hierarchy. Adding a site should not require an engineer.

3.6 Managing tenancy in the application and data layers

Concern	Approach
Tenant context	Resolved once per request or event batch and passed explicitly; never held in global state.
Connection routing	A small data-access component reads placement from the registry (cached) and returns the right connection, pooled or dedicated.
Schema changes	Applied by the pipeline to pooled and dedicated databases alike, using expand/contract migrations so old and new code both work during rollout.
Tenant lifecycle	Provisioning, active, suspended, offboarding, deleted. Suspension blocks access but continues ingesting, because the plant is still producing.
Metering and entitlement	Usage is attributed per tenant from day one, because whatever unit the product is sold in must be measurable at the isolation boundary. Entitlement is kept separate from metering, since the two change at different rates.
Offboarding	Designed now: export for the customer, deletion including archive, and a documented position on backups and on whether rollups require recomputation after a tenant leaves.

4. Security & Compliance

Security is presented as four boundaries, outside in, with an audit trail across all of them and the working assumption that any single control will eventually fail.

Boundary	Question	Principal controls
1. Getting in	Who and what can connect?	Outbound-only TLS from sites; per-collector X.509 identity issued at install; Front Door with WAF as the only public web entry; private endpoints for data services
2. Getting around	What can each identity do?	Federated customer identity; hierarchy-scoped roles; managed identities with least privilege between services; tenant context checked rather than inferred
3. Getting at the data	Can the wrong party read it?	Application tenant scoping with row-level security beneath it; encryption in transit and at rest; customer-managed keys where required
4. Getting it out	Can data leave by the wrong path?	Exports and APIs inside the tenancy model; egress logging; per-tenant quotas; controlled support access; deletion and offboarding designed in

4.1 Authentication and authorisation

Principal	Authentication	Authorisation
Collectors	Client certificate per collector issued at install, private key generated on the host and held in its certificate store or TPM; individually revocable	May POST telemetry and read its own configuration. The collector-to-asset mapping lives in the registry, so a collector cannot choose its tenant: the ingest API stamps the authenticated identity and ignores any tenant claim in the payload.
Customer users	OpenID Connect through Entra External ID, federated to the customer's identity provider, with MFA enforced there	Token validated at the API; tenant resolved; role and hierarchy scope checked for every resource
MOD staff	Corporate Entra ID with MFA and conditional access	Least-privilege Azure RBAC, time-bound elevation, no standing production access, all access audited
Services	Managed identities; no stored credentials	Roles scoped to the exact resource and action needed

4.2 Encryption

- **In transit:** TLS 1.2 or higher on every hop, including collector to cloud, browser to Front Door and service to database. On site, OPC UA sign-and-encrypt where the equipment supports it.
- **At rest:** platform encryption by default across storage, databases and messaging, with customer-managed keys in Key Vault offered where a contract requires the customer to hold the key.
- **At the edge:** the local buffer is encrypted, because the collector host sits in an environment we do not control.

4.3 Secret management

- **Eliminate first:** managed identities for service-to-service calls, certificates for devices, and OIDC federation for deployment pipelines, so most of the system holds no secrets at all.
- **What remains lives in Key Vault**, accessed by managed identity, with soft-delete, purge protection and access logging.
- **Rotation is automated where supported**, and certificate expiry is alerted well in advance, because an expired device certificate silently takes a site offline.
- **Secrets never appear** in source control, pipeline output, application logs or images; secret scanning runs in CI.

4.4 Secure communication between components

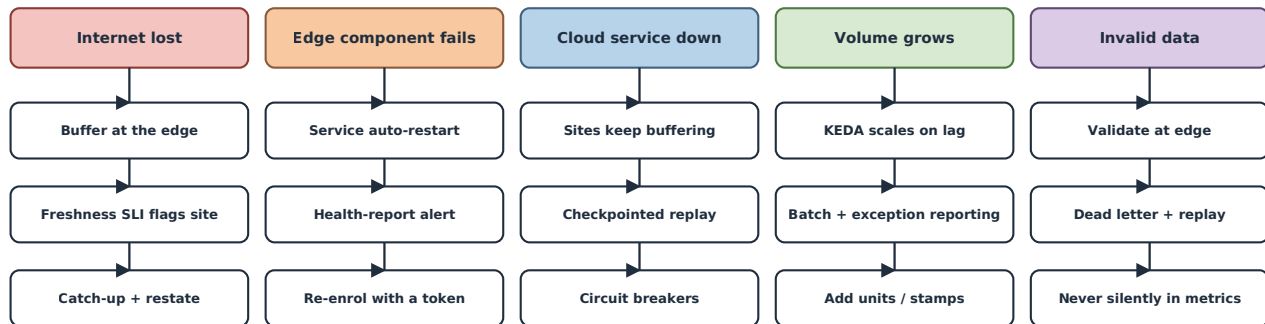
- Data services have public network access disabled and are reached over private endpoints inside a virtual network; workers and APIs run VNet-integrated.
- The public surface is exactly two things: Front Door for people, and the ingest and management endpoints for collectors, both fronted by Front Door with mutual TLS. They can be narrowed by IP filtering for customers with fixed egress addresses, and large customers can be given private connectivity over their own VPN or ExpressRoute. Where the optional command channel is enabled, collectors also reach the Service Bus namespace outbound on 443.
- Azure Policy enforces these rules so that drift is prevented rather than discovered later.
- On site, the collector opens no inbound port, runs under a least-privilege account, uses read-only credentials toward equipment, and never accepts an unsolicited instruction: anything it acts on, it asked for.

4.5 Customer access control and compliance posture

- Hierarchy-scoped roles with self-service user management, and a customer-visible audit log of sign-ins, exports, configuration changes and support access.
- Machine-to-machine integrations use per-tenant OAuth client credentials with scopes and rate limits, never shared user accounts.
- The design produces the evidence a SOC 2 or ISO 27001 programme depends on — infrastructure as code, reviewed changes, immutable artifacts, least privilege, audit trails and continuous posture assessment — without assuming such a programme exists today.
- Data residency is satisfied by per-tenant placement and regional stamps. Personal data is minimal; the sensitivity of the platform lies in production data, which customers treat as commercially confidential.
- The collector's behaviour (outbound-only, read-only, no listening ports) is what customers' OT security teams will review; a short security whitepaper describing it should be a first-class deliverable.

5. Reliability & Failure Handling

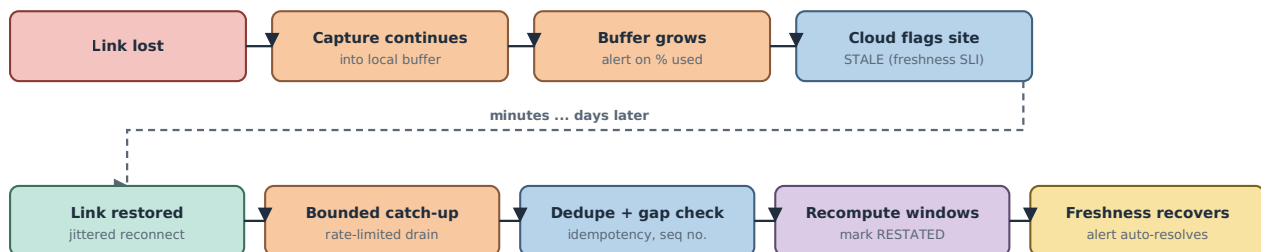
The dominant failure mode in this domain is not component failure but partial connectivity, so the first scenario below is a designed behaviour rather than an incident procedure.



Every path ends the same way: data is delayed or quarantined, never silently lost, and whatever the customer sees is labelled with its freshness and completeness.

Figure 5. The five required scenarios and their designed responses.

5.1 Internet connectivity is lost



Principle: connectivity loss is a FRESHNESS problem, never a DATA-LOSS problem —
as long as the outage is shorter than the buffer. Buffer size is therefore a business decision (see section 8).

Figure 6. Outage and reconnection flow.

Aspect	Behaviour
What happens	Collection continues. Events accumulate in the durable buffer. The forwarder retries with exponential back-off and jitter.
Detection	Missed health reports from the collector, plus the per-site freshness indicator crossing its threshold. Alerting is schedule-aware: a site that is not scheduled to produce is not an outage, so heartbeats are tracked separately from data.
Recovery	Jittered reconnection, then rate-limited catch-up so neither one returning site nor a whole region returning at once can flood the pipeline. Duplicates are dropped by idempotency key, gaps detected by sequence number, and rollups for affected windows are recomputed and marked as restated.
Customer experience	The dashboard says the site's data is delayed, then fills in, with restated periods flagged. A number is never silently frozen or silently changed.

Aspect	Behaviour
Limit	If an outage outlasts the buffer, the oldest data is lost. The gap is recorded explicitly so reports show no-data rather than implying zero production, and buffer size is sized against the worst credible outage (\$8).

Closing windows on time and restating visibly is a deliberate choice. Holding windows open would delay every customer's reporting because of the worst-connected site, and silent restatement costs more trust than visible lateness: a number that changes after someone has acted on it is worse than a number that was labelled late.

5.2 An edge component fails

Failure	Behaviour
Collector process crashes	The service restarts; the buffer survives because it is durable and written before acknowledgement; sequence numbers continue from the persisted value, so no duplicates or false gaps.
Host or disk fails	Data not yet forwarded is lost for that interval unless the source can be re-read. Missed health reports detect it. A replacement host is enrolled with a fresh one-time token and pulls its configuration from the management endpoint, so recovery does not depend on someone on site knowing the settings. A warm standby collector can be offered for critical sites.
A source or adapter fails	Other adapters continue (bulkheads per source). The affected assets are marked as no-data rather than stopped, and reconnection backs off so a struggling controller is not hammered.
Buffer approaching capacity	Alert on utilisation; if overflow is unavoidable, a documented policy keeps state changes and counters and downsamples high-frequency analog values first.
A bad collector release	Ring rollout limits exposure; the updater health-checks and rolls back automatically; the buffer format is versioned so the previous version can still read it.
Clock drift	Event time and observed time are separate fields; the collector reports its clock offset and excessive drift raises an alert, because drift on devices outside our control is a real source of corruption.

5.3 A cloud service becomes unavailable

Failure	Behaviour
Ingest API or log unavailable	Indistinguishable from connectivity loss at the site: the batch is not acknowledged, so the collector keeps it and retries. Nothing is lost while the outage is shorter than the buffer.
Processing workers down	Events remain in the stream within its retention; workers resume from their last checkpoint, and because every stage is idempotent, reprocessing is safe. Lag is alerted, and lag approaching retention is the true danger signal.
Database unavailable or slow	Circuit breakers stop calling a failing dependency, checkpointing pauses and backlog builds in the stream deliberately. Autoscaling is capped so that more workers cannot overwhelm a recovering database. Dashboards show last-known data with a staleness banner.
Live push or API degraded	Browsers fall back to periodic polling; the static front end is still served from cache.

Failure	Behaviour
Command channel unavailable	Nothing depends on it: configuration and versions still arrive by pull, and health still arrives on the uplink.
Region outage	Sites buffer throughout. Recovery redeploys the stamp from infrastructure code into another region and restores databases from geo-redundant backups. This is a slower recovery than a hot standby and is accepted at launch for cost reasons; a warm secondary region is a later step if customers require it.

Zone redundancy is enabled on the services that support it, so a single datacentre failure inside the region is absorbed without intervention.

5.4 Message volume grows significantly

Lever	Action
Reduce at the source	Report-by-exception and deadbands on analog values, batching and compression. State transitions and counters are never filtered out.
Ingest	Add units or move tier; when partitions become the constraint, route to a dedicated Event Hubs namespace with more partitions.
Processing	KEDA scales workers on lag up to the partition count; database writes are batched rather than row-by-row.
Storage	Rollup volume grows with asset count, not event rate, so it stays manageable. Raw storage is where volume hurts, and this is the point at which the deferred engine decision is made with real numbers.
Architecture	New stamps for new tenants; demanding tenants moved to dedicated resources.
Protection	Bounded queues everywhere, because an unbounded queue is a deferred outage; per-tenant quotas; and deliberate, prioritised shedding under sustained overload — analog detail is downsampled, state transitions are never dropped.

5.5 A customer site generates invalid or unexpected data

Control	Behaviour
Validation at the edge	Schema validation before buffering, with the raw value logged locally and counted in the health report, so the problem is visible where it originates.
Validation in the cloud	Revalidated on arrival; invalid or unattributable events are dead-lettered with a reason code. One bad message never blocks a partition.
Semantic checks	Counter resets and rollovers are handled explicitly rather than read as negative production; timestamps far in the future or past are flagged; implausible values are marked suspect; unknown tags are held as unmapped rather than dropped.
Schema evolution	Events carry a schema version, consumers tolerate additive change, and processors are deployed ahead of collectors that emit a new version.
Ownership and replay	Dead-letter depth per site is monitored and owned. After a fix, a replay tool re-submits events; idempotency makes replay safe, and affected rollups are restated.
Customer experience	A data-quality indicator per site and asset; support is notified; invalid data never flows silently into reported metrics.

5.6 Patterns applied throughout

Pattern	Where	Purpose
Store and forward	Edge buffer	Connectivity loss costs freshness, not data
Idempotency keys and sequence numbers	Edge through processing	Safe retries and replay; gap detection
Retry with exponential back-off and jitter	Forwarder and all outbound calls	Recover from transient faults without stampedes
Circuit breaker	Calls to databases and dependencies	Call a failing dependency less, not more
Bulkheads	Per adapter; separate compute for ingest, API and jobs	Contain failures
Bounded queues and back-pressure	Everywhere	Fail visibly rather than exhaust resources
Checkpointing	Stream consumers	Resume exactly where processing stopped
Dead letter with replay	Processing	Quarantine without losing data
Zone redundancy, backups, redeploy from code	Cloud platform	Datacentre and region failures

6. Operations & Support

6.1 Monitoring strategy

Four layers are monitored, and the one customers feel comes first. The business question is whether every site is current and whether a customer's number is derived from complete data — not whether the API is up.

Layer	Signals
1. Customer data health	Per-site freshness (age of the newest processed event); per-tenant completeness (accepted versus what collectors report sending, per interval); restatement rate; data-quality rate
2. Pipeline health	Ingest throttling and connected collector count; consumer lag per partition; processing errors; dead-letter depth and age; database latency and utilisation
3. Fleet health	Health-report recency, version, buffer utilisation, disk free, last successful read per source, clock offset, certificate expiry
4. Service health	API availability and latency, error rate, sign-in failures, live-connection counts

Reconciliation counts per site per interval — sent versus accepted — catch silent loss that per-event tracing cannot, because the absence of something never received leaves no trace. Metrics are kept deliberately low-cardinality, aggregated to site or tenant, with per-entity detail pushed into logs and traces; a metric labelled per source per signal per tenant is a combinatorial cost trap.

6.2 Alerting approach

- **Alert on symptoms, investigate causes.** A site stale beyond threshold while scheduled to produce, completeness below target, lag approaching retention, or a growing dead-letter queue are pages. Resource utilisation is a dashboard item.
- **Aggregate before alerting:** a regional outage should raise one incident, not two hundred site alerts.
- **Schedule-aware thresholds,** so planned shutdowns are not incidents.
- **Every alert has an owner and a runbook,** and alerts that never lead to action are removed. For a small team, alert fatigue is a reliability risk in itself.
- **Slow-burn alerts** for certificate expiry, buffer utilisation trending upward at a site, storage growth and cost anomalies.

6.3 Troubleshooting workflow

The common ticket is "yesterday's number for line 3 looks wrong". The workflow walks the data path backwards, from what the customer sees to the machine.

1. Check the freshness, completeness and restatement markers shown next to the number.
2. Check the collector's latest health report: last seen, version, buffer utilisation, last read per source, clock offset.
3. Compare reconciliation counts for the interval, which localises any loss to the edge or the cloud.
4. Check dead-letter entries and reason codes for that site.
5. Follow the correlation identifier from the edge batch through traces and processing logs, filtered by tenant and site.
6. Check configuration in the registry — asset mapping, rated speed, shift calendar — and recent changes in the audit log.

- 7. Fix, replay the affected events, let the rollups restate, and confirm with the customer.
- 8. If still unclear, request a diagnostic bundle from the collector over the optional command channel where the customer has enabled it (\$6.8); it uploads to tenant-scoped storage. No remote access into the customer's network is required at any point.

6.4 Deployment strategy and delivery pipeline

Delivery has two targets with very different risk profiles: the cloud platform, which MOD controls and can roll back in seconds, and the collector fleet, which runs where nobody from MOD can reach it. Both start from the same repository and the same build, and both progress through gates driven by health signals rather than elapsed time.

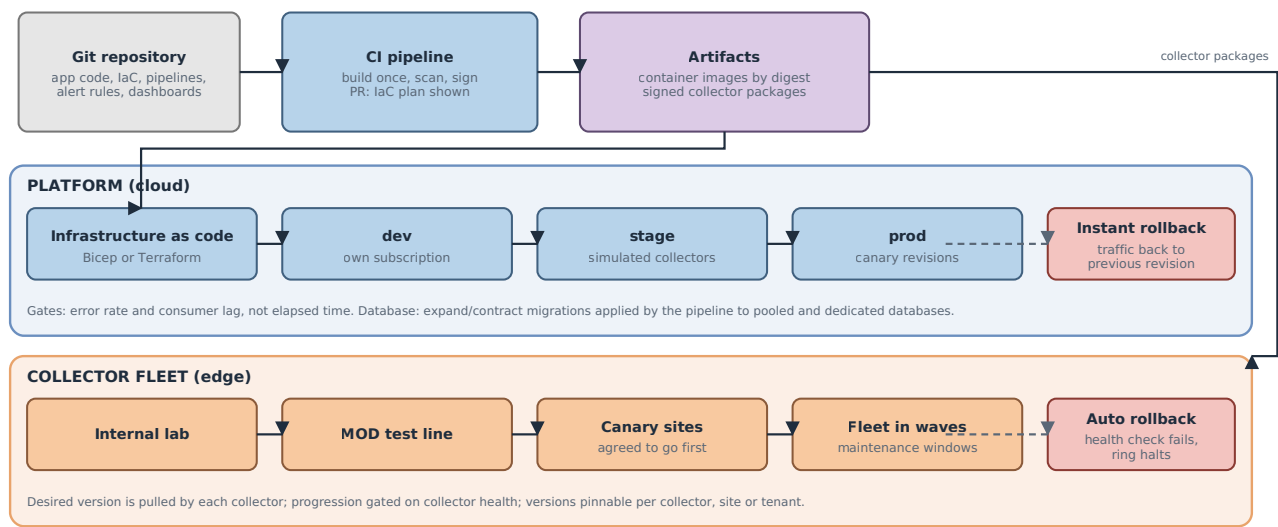


Figure 7. Delivery pipeline: one build, promoted by digest to the platform environments, and released to the collector fleet in rings.

Practice	How
Everything in Git	Application code, infrastructure, pipelines, alert rules and dashboards.
Infrastructure as code	Bicep or Terraform, following the team's existing choice. Bicep is Azure-only; Terraform needs secure remote state. Portal-driven changes are rejected.
Build once, promote	GitHub Actions or Azure DevOps, matching current tooling. CI builds and tests an image, scans dependencies, and the same digest moves through dev, staging and production, so what was tested is provably what deploys.
Reviewed infrastructure changes	A plan is shown on the pull request and applied by the pipeline; no manual portal changes, with policy detecting drift.
Environments	Separate subscriptions per environment; staging exercised by simulated collectors replaying recorded site traffic.
Progressive delivery	Container Apps revisions with traffic splitting; canaries gated on error rate and consumer lag rather than elapsed time; instant rollback.
Database changes	Expand/contract migrations applied to pooled and dedicated databases by the pipeline.

Practice	How
Collector releases	Signed packages; desired version served by the management endpoint and pulled by the collector; rings from internal lab to a MOD test line, to canary customer sites, to the fleet in waves; progression gated on collector health; upgrades respect site maintenance windows and the buffer covers the restart. We build and test the updater and the rollout mechanism ourselves, because unmanaged fleet upgrades are the largest operational risk.

6.5 Logging and diagnostics

- Structured logs via OpenTelemetry, always carrying tenant, site, correlation identifier and component version.
- Distributed tracing from the edge batch through to the rollup write, so a single number can be traced back to the events behind it.
- Sampling for high-volume successful traces, with all errors retained; verbose logs directed to cheaper tables; audit logs kept separately with longer retention and restricted access.
- No secrets, tokens or unnecessary personal data in logs.
- Edge logs rotate locally, with summaries in the health report and full bundles uploaded on demand.

6.6 Collector lifecycle

Collectors are the only components running where nobody from MOD can reach them, so their lifecycle is designed rather than left to field practice. The goal for installation is that a site technician needs one file and one code, and the goal for upgrades is that they are routine, reversible and cost no data.

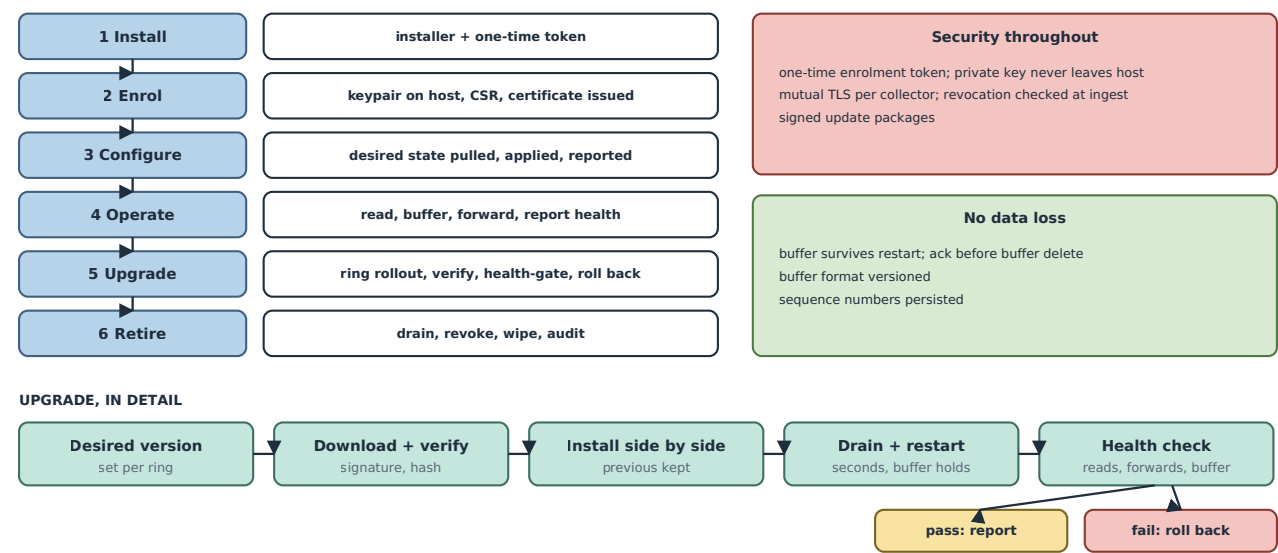


Figure 8. Collector lifecycle, with the upgrade sequence expanded.

Registration and enrolment

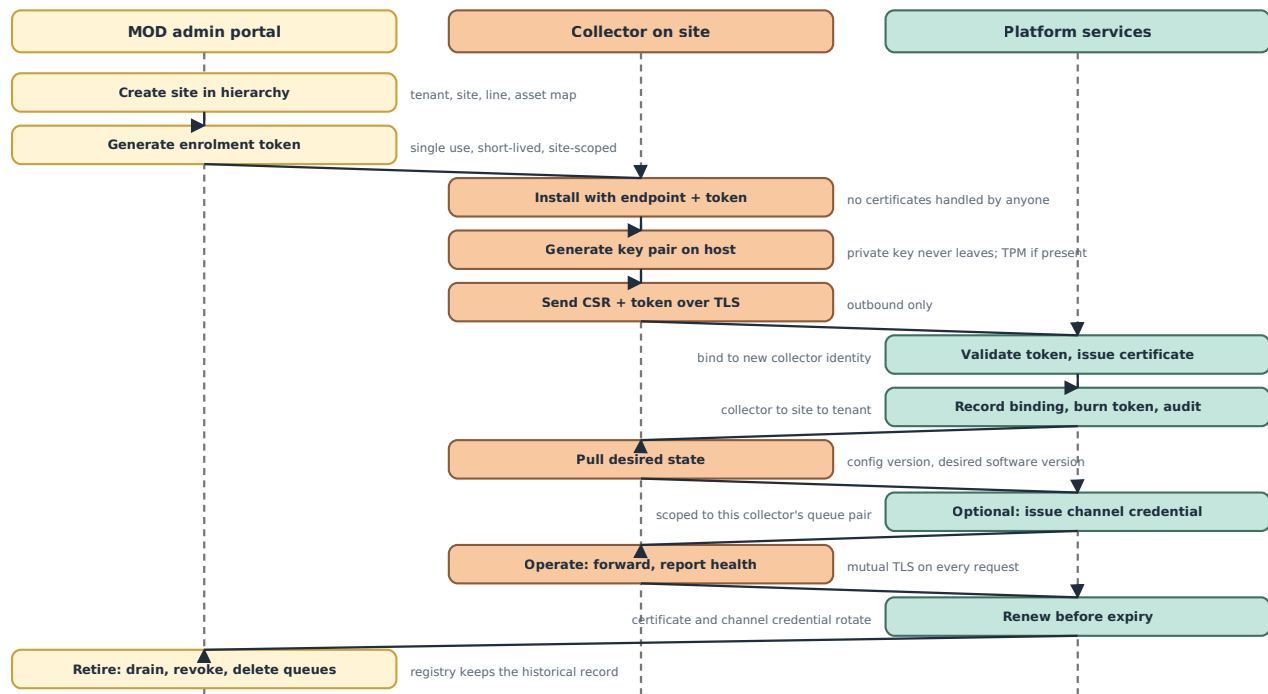


Figure 9. Enrolment, renewal and retirement: who generates what, and what is exchanged.

- 1. The site is created first, in the portal.** An administrator adds the site to the tenant hierarchy and generates a **one-time enrolment token**: short-lived, single-use, scoped to that tenant and site.
- 2. The installer asks for two things:** the cloud endpoint and that token. Nothing else — no certificates to copy, no connection strings, no firewall form.
- 3. The collector generates its own key pair on the host** and creates a certificate signing request. The private key never leaves the machine and is held in the operating system's certificate store, or in a TPM where one is available.
- 4. It posts the request with the token over TLS.** The platform validates the token, issues a client certificate bound to a newly created collector identity, records the collector-to-site binding in the registry, burns the token, and writes an audit entry.
- 5. First configuration arrives immediately,** and the collector begins reporting health. The site appears in the portal as online within seconds of install.

Stated as artefacts, because the security of the whole fleet rests on which party generates which secret and where it is allowed to travel:

Artefact	Generated by	Travels	Lifetime
Enrolment token	Platform, when an administrator creates the site	Once, to the installer, out of band (deployment ticket or install sheet)	Single use; expires in hours, not weeks
Collector key pair	The collector, on the host, at first run	Never. The private key stays in the host certificate store or TPM	Life of the collector; regenerated at renewal if policy requires
Certificate signing request	The collector	Once, outbound over TLS, together with the enrolment token	Moments

Artefact	Generated by	Travels	Lifetime
Client certificate	The platform's certificate authority	Back to the collector on the same call	Months, renewed automatically well before expiry
Collector identity record	The platform	Stays in the registry: identity, tenant, site and asset bindings, issued certificates, versions, status	Kept after retirement, because historical reports need the attribution
Command-channel credential (optional)	The platform, after enrolment	To the collector on request, over mutual TLS	Short-lived; renewed on the authenticated path (§6.8)

What is exchanged, in order: the installer is given the endpoint and the token; the collector sends its certificate signing request with that token; the platform returns the issued certificate, the collector identity and the first configuration document. Nothing else passes at enrolment, and no credential is ever carried by a person.

Renewal

- **Certificates renew automatically, well before expiry**, over the collector's existing authenticated connection: it generates a new key pair and submits a request signed with its current certificate. No token and no human involvement. If the renewal window is missed, the fallback is enrolment with a fresh token, which is deliberately a manual act.
- **Expiry is monitored centrally** and alerted well in advance, because a lapsed certificate takes a site offline silently and looks exactly like a network fault.
- **Configuration renews continuously by pull**; there is no separate refresh mechanism to fail.
- **Command-channel credentials are short-lived by design** and reissued on the same authenticated path, so a leaked one expires rather than persisting.

What keeps setup as simple as possible

Technique	Effect
One-time token instead of pre-issued certificates	Nothing secret is emailed, carried on a USB stick or reused across sites. A leaked token is single-use and expires.
Keys generated on the host	No key distribution problem at all.
Bulk token generation	A multi-site rollout gets one token per site from a single action, or a per-site code included in the deployment ticket.
Site templates	A new site inherits poll rates, batching, buffer sizing and naming conventions from a template, so the default configuration is already right.
Discovery wizard	After enrolment the collector browses reachable OPC UA endpoints and proposes an asset and tag mapping, which an engineer confirms in the portal rather than typing.
Approval gate	A collector may also self-present as pending, and an administrator approves it in the portal. Useful where the installer cannot be given a token in advance.
Preflight self-test	On first run the collector checks outbound connectivity, proxy behaviour, TLS interception, clock offset and disk headroom, and reports a readable pass or fail list. Most failed installs are network policy, and this surfaces that in minutes rather than after a week of missing data.

Technique	Effect
Unattended install	The same installer accepts a silent-mode answer file for customers with software deployment tooling.

Configuration

- **Desired state lives in the registry and is versioned.** It holds the source list and credentials reference, asset and tag mapping, poll or subscription rates, batching and buffer settings, and the desired software version.
- **The collector pulls on a short interval,** applies the change atomically, and reports back the configuration version and a hash of what it actually applied, so drift is visible centrally.
- **Last known good.** If a new configuration fails validation or its post-apply health check, the collector reverts to the previous generation and reports the failure. A bad configuration cannot strand a site.
- **Replacing hardware is a fresh enrolment;** the retired identity is revoked and the new one bound to the same assets.

De-registration and retirement

1. **Drain first.** The collector is marked retiring; it stops reading sources but continues forwarding whatever is still buffered, so a decommissioning does not discard hours of production history.
2. **Then revoke.** The certificate is revoked and the identity marked retired. The ingest API checks a cached deny list on every request and rejects a revoked collector outright.
3. **Uninstall wipes local state:** private key, buffer, cached configuration and local logs.
4. **Registry retains the record** — which assets it served and for what period — because reports over past periods still need that attribution. Certificates can also be revoked immediately, without draining, if a host is believed compromised.
5. **Everything issued is withdrawn together:** the client certificate is revoked, any command-channel credential is expired and the collector's queues are deleted (§6.8), and asset bindings are closed with an end date rather than deleted.
6. **Every step is audited** and visible to the customer.

Authentication and the security mechanisms in use

Mechanism	What it protects against
One-time, short-lived, site-scoped enrolment token	An installer package or a leaked code being used to register rogue collectors
Key pair generated on the host; private key in the OS store or TPM	Key theft in transit and key reuse across machines
Mutual TLS per collector on every request	Anonymous or spoofed senders; gives per-collector revocation
Identity stamped server-side onto every event	A compromised collector claiming another customer's tenant
Revocation list checked at the ingest API	A stolen host continuing to send after decommissioning
Automatic certificate renewal before expiry, with expiry alerts	A site silently going dark because a certificate lapsed
Endpoint pinning and allow-listed outbound destinations	A collector being pointed at an attacker's endpoint

Mechanism	What it protects against
Code-signed update packages, signature verified before install	A malicious or corrupted update reaching the fleet
Encrypted local buffer, least-privilege service account, no inbound ports, read-only equipment credentials	The collector becoming a foothold into the plant network
Audit trail of enrolment, configuration, upgrade and revocation	Undetected changes to the fleet

Upgrades without downtime or data loss

Upgrades are driven by desired version, not by anyone visiting a site. Rings progress from an internal lab to a MOD test line, then canary customer sites that have agreed to be first, then the fleet in waves, with progression gated on fleet health rather than elapsed time (§6.4).

Property	How it is achieved
Nothing is lost while the collector restarts	Capture stops for seconds, but the buffer is durable on disk and the sources are re-read from their last acknowledged position on start. For sources that cannot be re-read, the gap is bounded by the restart and is recorded rather than hidden.
Nothing is lost in flight	A batch is only removed from the buffer after the cloud acknowledges it, so an upgrade during a send simply results in a retry — and idempotency keys make the retry harmless.
The new version can read the old buffer	The buffer format is versioned, and each release must read the previous format. This is a release gate, tested automatically.
No duplicates or false gaps after restart	Sequence numbers are persisted with the buffer, not held in memory.
The switch is atomic	The new version installs side by side; the service is stopped, the symlink or service binding is switched, and the service is started. The previous version stays on disk.
Upgrades avoid disrupting operators	Each site has a maintenance window; the collector applies a new desired version only inside it, unless the release is marked urgent.

Rollback

- **Automatic triggers:** the post-upgrade health check fails, the process crash-loops, the new version cannot read the buffer, the configured sources stop yielding data, or no successful forward occurs within a grace period.
- **What happens:** the updater restores the retained previous version and its configuration generation, restarts, verifies health, and reports the rollback with a reason code. The buffer is untouched throughout.
- **Fleet effect:** a rollback in a ring halts progression of that release automatically and raises an alert, so one bad build stops at the canary ring rather than reaching hundreds of sites.
- **Manual rollback** is the same mechanism: set the desired version back. Versions can be pinned per collector, per site or per tenant, which doubles as a kill switch.
- **If a collector cannot recover at all** — corrupted install, failed disk — the recovery path is a fresh install and re-enrolment, which is why enrolment is designed to be a two-field operation.

6.7 The MOD administration portal

Everything MOD staff do to customers and collectors happens in one audited place, against the control plane. The portal holds no data of its own: through the Admin API it edits the registry and the desired-state documents

that the rest of the platform reads. Staff reach it through Front Door and sign in with corporate Entra ID (Figure 1).

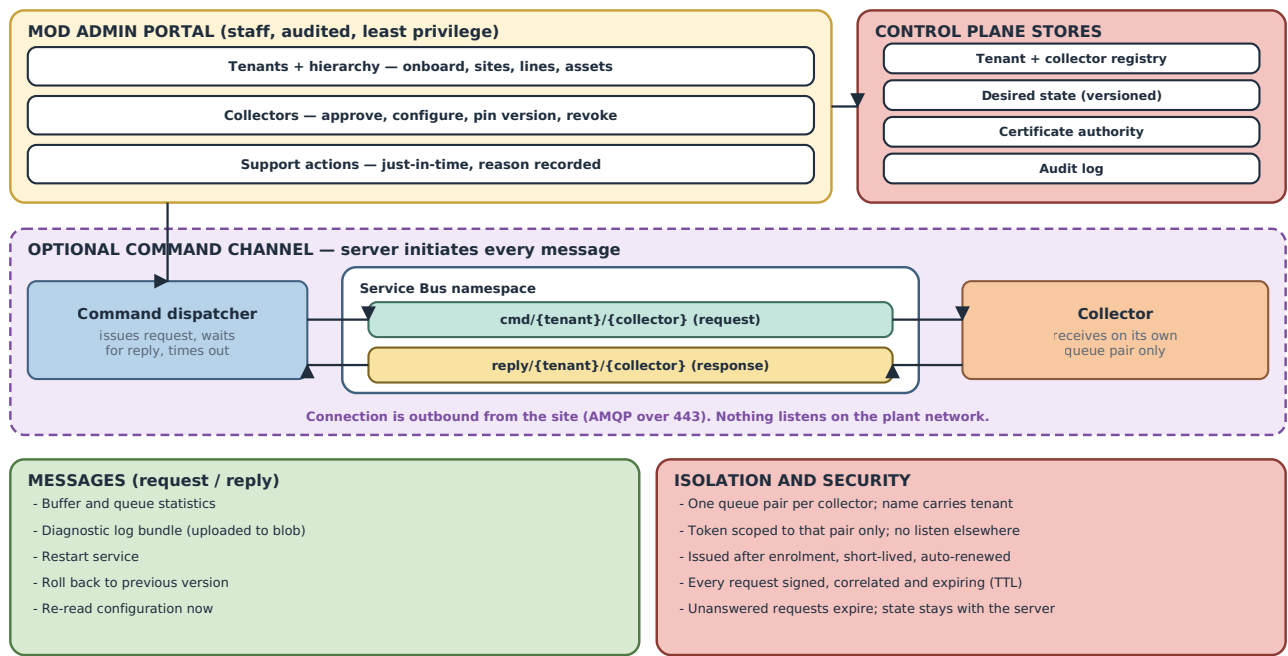


Figure 10. The administration portal, the control-plane stores, and the optional command channel.

Area	What an administrator can do
Customer onboarding	Create a tenant; set isolation mode (pooled or dedicated) and regional placement; build the hierarchy of group, site, line and asset; configure federation with the customer's identity provider
Site setup	Create a site, generate its one-time enrolment token, apply a configuration template, and watch the collector appear as it enrolls
Collector administration	Approve a pending collector; view identity, certificate status, version, health and buffer history; edit desired configuration; pin or advance the software version; move it between release rings; revoke it
Fleet operations	Filter the fleet by tenant, site, version, health or certificate expiry; apply a change to a selected group; watch a release progress ring by ring and halt it
Data operations	Inspect dead-lettered events and reason codes, trigger a replay, and see which rollups were restated as a result
Support access to customer data	Time-bound elevation with a recorded reason, scoped to one tenant, fully audited and visible to that customer
Evidence	Read the audit trail: who changed what, when, and on whose authority

What the portal deliberately cannot do: reach into a customer network, open a session on a collector host, or write anything to production equipment. Its effect on a site is limited to changing desired state that the collector later pulls, or — where the optional channel is enabled — sending a request the collector chooses to answer on a connection it opened itself.

6.8 Optional command channel

Pull-based configuration is enough to run the fleet, but it is slow when an engineer is on the phone with a customer: every question waits for the next poll, and there is no way to ask for something the collector was not already reporting. The optional command channel closes that gap without weakening the network posture. It is shown with purple dashed lines in Figure 1.

Shape

- **Service Bus, one queue pair per collector:** a request queue and a reply queue, both named with the tenant and collector identity. The pair is created when the collector is approved and deleted when it retires.
- **Every message is initiated by the server.** The collector never sends anything on this channel that is not an answer to a request; health and telemetry continue on the ordinary uplink.
- **The network direction is unchanged.** The collector holds an outbound connection to Service Bus over AMQP on 443 and receives on it. Nothing listens inside the plant, no port is opened and no firewall exception is needed — the same posture as the data path.
- **Request and reply are correlated,** and the dispatcher in the Admin API owns the pending request, its deadline and its result.

Security and per-customer isolation

Control	Effect
One queue pair per collector, named by tenant and collector	No shared queue exists that could carry one customer's traffic to another customer's collector
Credential scoped to that pair alone	A collector may receive on its own request queue and send on its own reply queue, and has no rights anywhere else in the namespace — not to list, not to peek, not to subscribe
Issued only after enrolment, over mutual TLS	The channel inherits the identity established at enrolment instead of introducing a second, weaker way in
Short-lived, automatically renewed	A leaked credential expires on its own; revocation is immediate by refusing renewal and deleting the queue
Payloads signed by the platform, verified by the collector	A message that did not originate from the platform is rejected even if the transport were somehow compromised
Allow-listed command types only	The collector executes a fixed, small set of requests and rejects anything else. There is no general-purpose remote execution, ever
Every request and reply audited	Who asked what, of which collector, when, and what came back — visible to the customer for their own sites
Encrypted in transit and at rest on the broker; no retention beyond delivery	Message content does not accumulate anywhere

Timeouts and failure semantics

- **Every request carries a deadline and a time-to-live on the queue,** both configured per command type. A request not collected before its time-to-live expires is removed rather than delivered hours later to a site that has just come back.
- **The dispatcher times out independently of the broker,** so an operator always gets an answer: the reply, or a clear statement that the collector did not respond within the deadline.
- **A disconnected site is a normal outcome, not an error.** The request expires and the portal reports the site as unreachable, which is information in itself.

- **Requests are idempotent or explicitly single-shot.** Asking twice for statistics is harmless; a restart or a rollback carries a request identifier the collector records, so redelivery cannot execute it twice.
- **Nothing depends on the channel.** If Service Bus is unavailable, or the channel is disabled for a customer, the platform behaves exactly as it does without it: configuration and versions still arrive by pull, health still arrives on the uplink.

What it is used for

Request	Why it earns a channel rather than a poll
Buffer and queue statistics on demand	Answers whether a site is catching up, during a live support call, in more detail than the routine health report carries
Diagnostic log bundle	The collector gathers logs, configuration and buffer statistics and uploads them to tenant-scoped storage with a scoped, short-lived credential. Logs never travel through the queue
Restart the service	Clears a stuck adapter without asking the customer to find the machine
Roll back to the previous version	Recovers a site that a release has degraded, immediately rather than at the next poll
Re-read configuration now	Applies an urgent change without waiting out the polling interval

NOTE

The channel is optional per customer and per site, because some security teams will accept an outbound data connection but no command path of any kind. The platform must remain fully operable without it, which is why configuration and version management are pull-based in the first place: the channel adds speed, not capability.

7. Cost Considerations

7.1 Major cost drivers

Driver	Billed as	Grows with
Raw data retention	Storage per GB-month, plus analytics engine compute if one is used	Event rate multiplied by retention period — the dominant line in a multi-year product
Ingest	Throughput units with a daily ingress allowance, metered in fixed-size blocks	Message count and size; number of sites
Compute	vCPU and memory seconds	Event volume, API traffic, background jobs
Relational storage	Compute tier, storage and backups; elastic pools share capacity	Tenants, rollup volume, dedicated databases
Observability	Log and metric ingestion and retention per GB	Verbose logging, high-cardinality metrics and traces at volume — frequently the surprise bill
Networking	Egress, private endpoints, edge requests	Exports, integrations, multi-region
Fixed platform services	Front Door, WAF, security tooling, Key Vault	Environments rather than usage
Engineering time	People	Every self-operated component and every dedicated tenant

7.2 Where costs grow over time

- **Retention accumulates:** stored data grows every month even with a flat customer count, unless tiering and deletion are automatic.
- **Logs and metrics grow faster than traffic** if nobody owns them.
- **Dedicated tenants** each add a floor cost and operational effort.
- **Catch-up spikes** after widespread outages temporarily multiply ingest and processing.
- **Residency and multi-region** duplicate the fixed cost of a stamp.
- **Per-message ingest pricing compounds at telemetry volume**, which is why the crossover point with a dedicated event stream should be tracked rather than discovered.

7.3 Design decisions that balance scalability and cost

Decision	Cost effect
Report-by-exception, batching and compression at the edge	Fewer and smaller messages; lower ingest and storage
Rollups as the read model	Cheap, fast dashboards; raw data can be retained briefly
Three storage tiers with lifecycle rules	Expensive storage only for hot data; the long tail lives in an open, cheap format
Raw engine deferred until measured	Avoids paying for always-on analytics capacity before it is justified

Decision	Cost effect
Pooled by default, dedicated by exception	Shared capacity, with dedicated resources only where a contract pays for them
Scaling on lag, scale to zero outside production	Pay for work rather than idle capacity
One shared log partitioned by collector, rather than a stream per tenant	Shared throughput units instead of a fixed cost floor per customer
Resource tagging and per-tenant metering from day one	Cost attribution cannot be retrofitted, and it informs pricing
Logging budget: sampling, low-cardinality metrics, cheaper log tiers	Controls the most common runaway bill
Reservations once a baseline exists; budgets and anomaly alerts	Discounts on steady workloads; problems found in days, not at month end

7.4 Sizing method

Real figures need measured event rates, which is an open question (§8). The method, however, can be stated now: volume is driven by **sites x assets x tags x change rate**, and the design reduces that number before it becomes expensive. Using the illustrative figures of §7.5 — about 130 sites, 400 lines and 64,000 tags, one change per tag every ten seconds — raw volume is on the order of hundreds of millions of values per day, while minute-grain rollups are on the order of a few million rows per day: roughly two orders of magnitude smaller. That ratio, rather than any price list, is what makes multi-year retention affordable and what justifies keeping raw data hot only briefly.

One practical detail worth deciding early: because ingest is metered in fixed-size blocks, batch size and compression at the edge change the metered message count materially. Batching should be tuned against the freshness target rather than left at a default.

7.5 Estimated ingest cost at installed-base scale

Assume an installed base of hundreds of production lines across many countries: 400 lines at roughly 130 sites, eight assets per line and twenty tags per asset — about 64,000 tags — with one collector per site batching every few seconds. The ingest layer then costs the following. Figures are list-price estimates and must be confirmed before they are quoted.

Scenario	Change rate	Sustained ingress	Throughput units	Indicative monthly
Exception reporting (proposed)	1 change per tag per 10 s	~0.4 MB/s	2 (with headroom)	~\$45
Moderate	1 change per tag per 2 s	~2 MB/s	3 to 4	~\$90
Polled, unfiltered	1 value per tag per second	~6.4 MB/s	8	~\$185, plus storage overage beyond about a day of retention

Per-event ingress charges are two to five dollars a month in every case, because batching collapses the metered event count: ingress is metered in 64 KB blocks, so a batch counts once until it exceeds that size. The throughput-unit line is effectively the whole bill. Two decisions move it materially: retention beyond roughly a day at high volume exceeds the storage allowance included per unit, and the managed capture feature costs more per unit than the stream itself, which is why the cold tier is written by our own worker instead.

A realistic first two years is smaller still. The installed base runs on perpetual licences and will not migrate at once, so a pilot through early production — five to thirty sites — sits at the floor of one to two throughput units, roughly twenty-five to forty-five dollars a month. At no plausible scale is the log a significant cost line: it is an order of magnitude below the relational tier. The comparison that does matter is with a managed device gateway, whose 4 KB metering puts the same traffic at roughly \$250 to \$500 a month — five to ten times the cost, bought for fleet features the requirements do not include (\$2.2).

NOTE

Retention tiers should be decided alongside the pricing model rather than after it, and whatever unit the product is sold in — per asset, per line, per site or per enterprise — must be measurable at the tenant boundary from the first release.

8. Open Questions & Assumptions

Each assumption below is stated with the decision it supports and what would change if it proves wrong. Several of these can change a decision rather than merely refine it.

8.1 Assumptions made

#	Assumption	Decision it supports	If it is wrong
A1	A small host can run on each site's network at the site-operations level or in the industrial DMZ, with local disk.	Edge collector with durable buffer	If no software is permitted: read from a system the site already runs, or accept scheduled exports; freshness suffers.
A2	Outbound HTTPS on 443 is permitted, possibly through a proxy, without inspection of our traffic.	Outbound transport with certificate authentication	TLS inspection breaks certificate-based collector authentication: proxy exclusions, a token-based fallback, or private connectivity per customer.
A3	Controller data is reachable read-only through OPC UA, native protocols, a gateway or an existing historian, without program changes.	Pluggable adapters, OPC UA first	More native drivers, or a partnership with a gateway vendor.
A4	Data is machine states, counts, speeds, faults and process values at roughly second-level granularity, not high-frequency waveform data.	Event pipeline and storage sizing	High-frequency signals require feature extraction at the edge and a different storage and cost design.
A5	Near real time means seconds to tens of seconds end to end, not milliseconds.	Batching, stream processing, push updates	Sub-second closed-loop requirements stay on site; the cloud is the wrong place for them.
A6	The platform only reads; it never writes to control systems. The only downlink is collector configuration.	Security model and outbound-only design	Any write path to equipment requires a separate safety and security design and customer OT approval.
A7	The worst credible outage is multiple days; buffer sizing starts from roughly a week and is refined with field data.	Buffer sizing and host specification	Longer outages need larger local storage or a documented downsampling policy.
A8	Customers range from single-site to multinational, and site count grows into the hundreds.	Bridge tenancy, stamps, fleet management	A handful of very large customers would justify more dedicated deployments and simpler ingest.
A9	One region at launch is acceptable, with residency needed later.	Single stamp with placement designed in	Day-one residency means two regional stamps from the start and a higher fixed cost.
A10	Microsoft stack and a small in-house team that both builds and runs the platform.	Managed services throughout	A larger or platform-specialised team could justify more self-managed components.
A11	Raw data must stay queryable for weeks; rollups for years.	Three-tier storage design	Long raw retention changes the raw store choice and the per-tenant cost floor.

#	Assumption	Decision it supports	If it is wrong
A12	The relationship between this platform and the existing on-premises product is undecided; the design works either as an extension of it or as a new collector.	Canonical event model and pluggable adapters	If the existing product becomes the edge producer, §1.5 becomes an adapter and forwarder added to it, and migrating installed-base history becomes a first-class requirement.
A13	The platform must manage collector configuration, versions and health remotely. The requirements do not state this; it follows from the requirements for small-team supportability, built-in diagnostics and modern maintenance practices.	The management endpoint, health reporting and ring-based updates	If field services manage collectors manually at install, the management endpoint shrinks to health reporting only. If a genuine downlink requirement appears, a managed device platform becomes the better choice and §2.2 is reopened.
A14	No path that writes to production equipment is required, now or later. The optional command channel (§6.8) carries operational requests to the collector only — statistics, logs, restart, rollback — and never anything that reaches a controller.	One-way data posture; pull-based configuration; an optional, allow-listed, outbound-connected command channel	Any requirement to send setpoints, schedules or acknowledgements toward equipment changes the security model fundamentally and needs customer OT approval before it is designed.
A15	Idempotency keys can be retained for at least the longest outage the buffer is sized for.	Deduplication of a backlog arriving long after the events were generated	A shorter key window means a returning site's events could be reprocessed as new; the window and the buffer size must be decided together.

8.2 Information needed from business stakeholders

Question	Why it changes the design
What is the commercial unit: per asset, per line, per site or per enterprise?	Defines the metering boundary and therefore how ingestion attribution and tenancy are structured.
Are the first customers new sites, or the existing installed base?	Installed base makes firewall traversal, co-existence and historical backfill the first hard problems; new sites make onboarding automation the priority.
Is this platform a replacement for, a companion to, or a hybrid with the current on-premises product?	Determines whether the existing product becomes the edge producer, and whether two data models can be avoided.
What availability, freshness and support commitments will be sold?	Drives redundancy, disaster-recovery investment and the on-call model.
What retention do customers expect, and what are they owed on exit?	The dominant cost driver, and the offboarding design.
Which jurisdictions and regulated industries must be served, and when?	Determines whether regional stamps are day-one or deferred.
Will some customers require dedicated infrastructure or customer-managed keys, and will they pay for it?	Sizes the dedicated path and its operational load.
Will customers accept the optional command channel, and on what terms?	Decides whether the channel is built early or left for later; the platform works either way.

Question	Why it changes the design
Who installs and supports collectors: MOD field services, customer IT, or partners?	Shapes installer tooling, documentation and support tiers.
Is there appetite for native integration with equipment manufacturers' own platforms?	Could turn adapters into a product line rather than a cost line.
What is the timeline for a first release, and which sites could be design partners?	The first increment depends on real sites (Appendix B).

8.3 Technical unknowns that could significantly change the design

Unknown	What it could change
Real event rates and payload sizes per site	Raw store engine, ingest tier, partition counts, the whole cost model
Distribution of equipment protocols across the installed base	Adapter priority; whether gateway partnerships are needed
Customer network realities: proxies, TLS inspection, permitted ports, isolated sites	Transport, authentication method, need for file-based or private paths
Actual connectivity quality and observed outage durations	Buffer sizing, catch-up rate, freshness thresholds
State of the existing codebase and how it talks to equipment today	Whether edge code can be reused, and the size of the migration effort
Existing Azure footprint, landing zone, identity and CI tooling	Infrastructure-as-code choice, pipeline choice, how much platform work is day zero
Team size, skills and on-call model	How much operational surface the architecture can justify, including the raw store choice
Clock synchronisation at sites	How far event time can be trusted and whether correction logic is needed
Volume and format of history to import, if the installed base is in scope	Backfill tooling and restatement at scale
Which identity providers customers use	Federation effort per onboarding

Four of these can only be settled at a real site, through its real network: whether the obtainable data is sufficient, what the true event rate is, whether a customer's security policy permits an outbound agent at all, and how bad connectivity really is. Those four answers move the storage engine, the buffer sizing, the transport design and the fleet strategy, and none of them can be settled by reasoning. That is the argument for the first increment in Appendix B.

Appendix A: Decision record

Each entry records the decision, the alternative, and the consequence accepted, so that a reader who disagrees can argue with the reasoning rather than guess at it.

#	Decision	Alternative	Consequence accepted
D1	Durable buffer at the edge, outbound-only connection	Direct streaming with no local store	Software and disk to manage on customer premises
D2	Compute derived metrics centrally	Compute at the edge and ship results	Higher bandwidth and storage
D3	Ship canonical events rather than pre-aggregated data	Aggregate at the edge to save bandwidth	Bandwidth cost, revisited if measured rates make it untenable
D4	Thin ingest API in front of Event Hubs, with collector management pulled from a separate endpoint	A managed device gateway providing identity, twins and updates	We own enrolment, certificate lifecycle and update rollout, in exchange for a one-way posture that matches the requirements and roughly a fifth to a tenth of the ingest cost at installed-base volume
D4b	One capped-batch submission method for both live traffic and catch-up	A separate bulk-upload path for large backlogs, and concurrent batches in flight per collector	Slower drain on high-latency links and more per-request overhead when batches are part-filled, in exchange for one arrival surface, one dedupe store and one replay story (§2.3)
D4c	An optional server-initiated command channel on Service Bus, one queue pair per collector	Pull-only management for everything, or a general-purpose remote-access tool	A second optional path to secure and audit, in exchange for answering support questions in seconds rather than at the next poll; the platform stays fully operable without it
D5	Partition by source, never by tenant	Partition by tenant, the intuitive choice	No cross-asset ordering guarantee, which the domain does not require
D6	Container Apps with KEDA, scaled on consumer lag	AKS for control, or Functions for simplicity	Less control than a cluster; a migration path exists
D7	Defer the raw-store engine pending measured rates	Commit now to one engine	The projection layer must keep the raw store swappable
D8	Pooled tenancy by default, dedicated by exception	Silo everything, or pool everything	Both paths must be built and tested; the application stays tenancy-mode agnostic
D9	Close aggregation windows on time and restate visibly	Hold windows open to absorb late data	Aggregates are not immutable, and the product must express restatement
D10	Single region at launch, placement as a control-plane property	Multi-region from day one, or ignore residency	Slower regional recovery initially; retrofitting placement later would be a rewrite

Appendix B: Suggested first increment

Prove the riskiest path end to end before building breadth. The first thing to ship is one real site, end to end, through its actual network — not the most demonstrable slice.

Phase	Scope	Exit criteria
0. Discovery	Two or three design-partner sites: capture network rules, protocols and measured event rates; agree the canonical schema	Assumptions in §8 replaced by measurements
1. Thin slice	One adapter, one transport, one tenancy mode, one dashboard: collector with buffer, ingest, one processing pipeline, rollups, freshness monitoring, infrastructure as code and CI/CD from day one	One real site live through its real network, with a deliberate outage test passed
2. Harden	Dead letter and replay, restatement in the UI, ring-based collector updates, reconciliation counts, alerting and runbooks, security review	A handful of sites stable for weeks; first customer security review passed
3. Broaden	More adapters, the raw-store decision made with real data, exports, tenant self-service administration, metering, the optional command channel	Onboarding a site takes hours rather than weeks
4. Scale	Additional stamps, dedicated tenancy option, regional residency, integration API	Growth is handled by configuration rather than redesign

Held correct regardless of scope cuts: the canonical event schema and the tenant boundary. Those are the two things a later rewrite cannot fix. **Deferred explicitly:** multi-region, self-service onboarding, configurable reporting, metering beyond the minimum, and anything predictive.